



Sichuan University -
Pittsburgh Institute

ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute





Topics

1. Fixed-point Numbers
2. Floating-Point Number

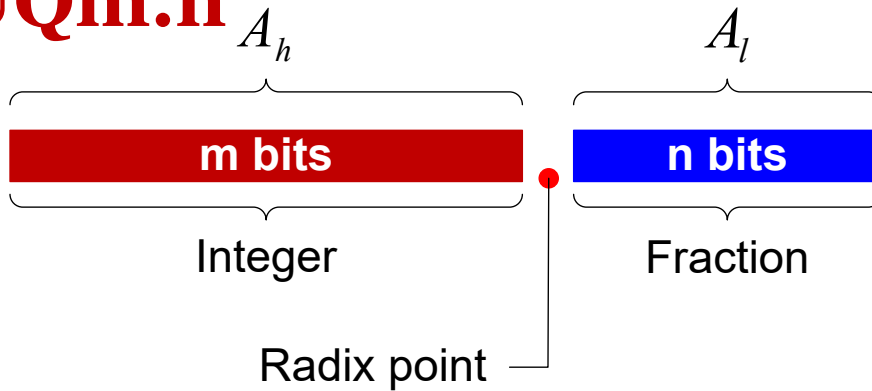
Fixed-point Numbers

$$\begin{aligned} & \mathbf{01101.101}_2 \\ &= 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3} \\ &= \mathbf{13.625} \end{aligned}$$

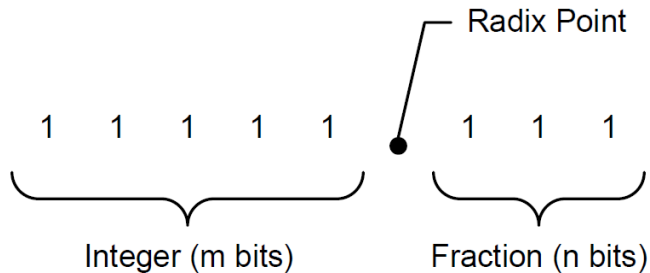
Unsigned Fixed-point Representation

UQm.n

Quantization Format



$$f = A_h + A_l \times 2^{-n}$$



$$\begin{aligned} 10101.101_2 &= A_h + A_l \times 2^{-3} \\ &= 21 + 5 \times 2^{-3} \\ &= 21.625 \end{aligned}$$

Unsigned Fixed-point Representation

UQm.n

Example: Convert $f = 3.141593$ to unsigned fixed-point UQ4.12 format.

- Calculate $f \times 2^{12} = 12867.964928$
- Round the result to an integer, $round(12867.964928) = 12868$
- Convert the integer to binary: $12868 = 11_0010_0100_0100_2$
- Organize into UQ4.12: $0011.0010_0100_0100_2$
- Final result in Hex: **0x3244**
- Error: $\frac{12868}{2^{12}} - f = -8.5625 \times 10^{-6}$

Convert

- The number **e**, also known as Euler's number, is a mathematical constant approximately equal to **2.71828** that can be characterized in many ways. It is the base of natural logarithms. It is the limit of $(1 + 1/n)^n$ as n approaches infinity, an expression that arises in the study of compound interest.
- **Convert e to unsigned fixed-point UQ4.12 format. (write in hex)**

Signed Fixed-point Representation

Qm.n



$$A = -1 \times b_{N-1} \times 2^{N-1} + \sum_{i=0}^{N-2} (b_i \times 2^i)$$

$$f = \frac{A}{2^n}$$

where $N = m + n + 1$

Signed Fixed-point Representation

Qm.n

Example: Convert $f = -3.141593$ to signed fixed-point Q3.12 format.

- Calculate $f \times 2^{12} = -12867.964928$
- Round the result to an integer, $round(-12867.964928) = -12868$
- Convert the absolute integer to binary: $12868 = 11_0010_0100_0100_2$

(Note that the integer is represented in two's complement.)

- Make the result into 16 bits: $0011_0010_0100_0100_2$
- Find the two's complement: $1100_1101_1011_1100_2$
- Final result in Hex: **0xCDBC**

- Error: $-\frac{12868}{2^{12}} - f = 8.5625 \times 10^{-6}$

Convert

- Convert - 2.71828 to signed fixed-point Q3.12 format. (write in hex)

Fixed-point Addition and Subtraction in UQ16.16 or Q15.16

Assume UQ16.16 $f_C = f_A + f_B$

$$\begin{cases} I_A = f_A \times 2^{16} \\ I_B = f_B \times 2^{16} \\ I_C = f_C \times 2^{16} \end{cases} \rightarrow \begin{cases} f_A = I_A \times 2^{-16} \\ f_B = I_B \times 2^{-16} \\ f_C = I_C \times 2^{-16} \end{cases}$$

$$\begin{aligned} \rightarrow f_C &= f_A + f_B \\ &= I_A \times 2^{-16} + I_B \times 2^{-16} \\ &= (I_A + I_B) \times 2^{-16} \end{aligned}$$

$$\rightarrow I_C \times 2^{-16} = (I_A + I_B) \times 2^{-16}$$

$$\rightarrow I_C = I_A + I_B$$

Fixed-point Addition and Subtraction in UQ16.16 or Q15.16

$$f_C = f_A + f_B \longleftrightarrow I_C = I_A + I_B$$

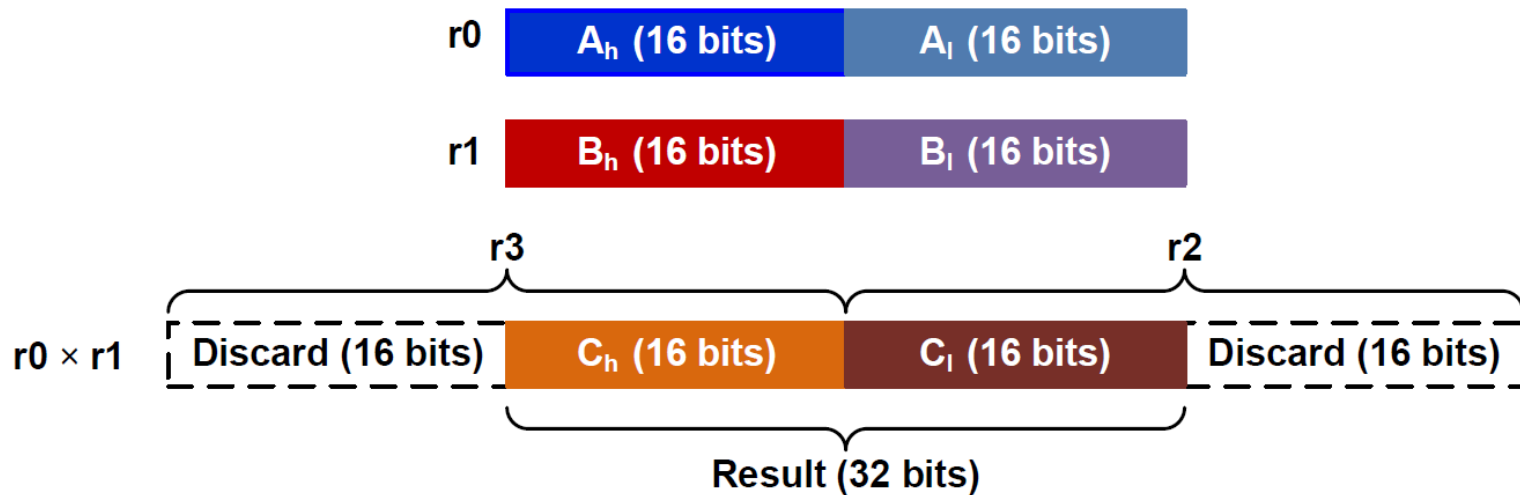
```
; r0 = fixed-point number fa  
; r1 = fixed-point number fb  
; r2 = fixed-point number fc
```

```
add    r2, r0, r1      ; fc = fa + fb  
sub    r2, r0, r1      ; fc = fa - fb
```

Fixed-point Multiplication in UQ16.16 or Q15.16

$$\begin{aligned}
 f_C &= f_A \times f_B \\
 &= (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \\
 &= (I_A \times I_B) \times 2^{-32}
 \end{aligned}
 \quad \left. \vphantom{\begin{aligned} f_C &= f_A \times f_B \\ &= (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \\ &= (I_A \times I_B) \times 2^{-32} \end{aligned}} \right\} I_C = (I_A \times I_B) \times 2^{-16}$$

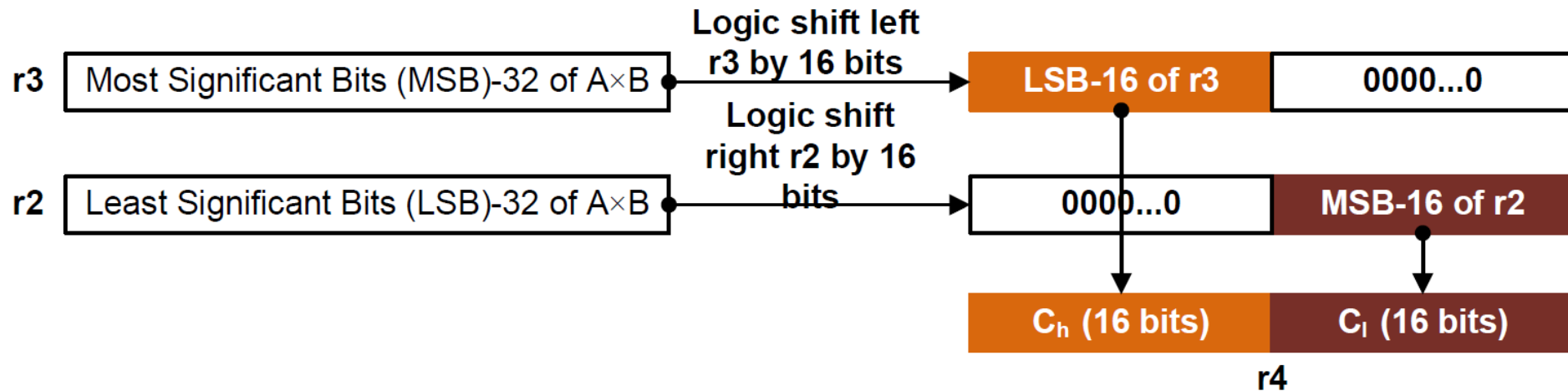
$$f_C = I_C \times 2^{-16}$$



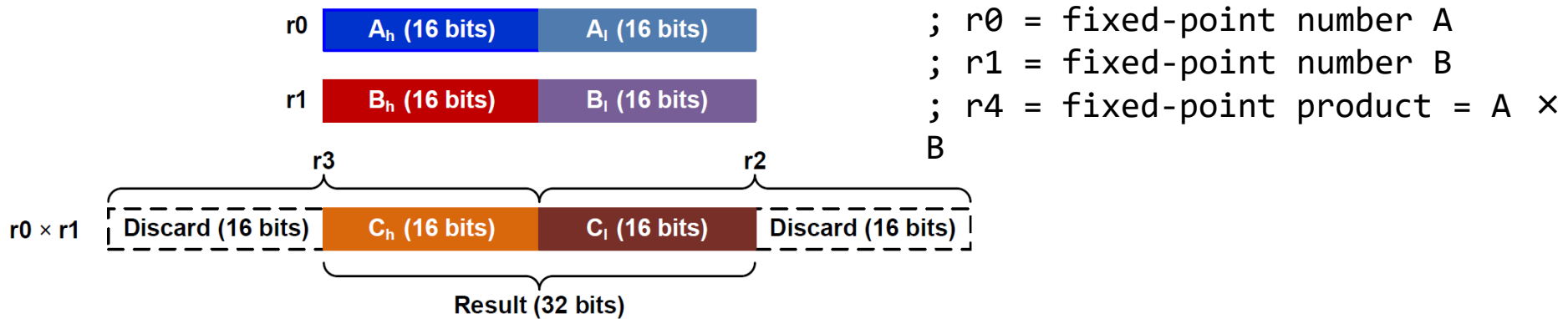
Fixed-point Multiplication in UQ16.16 or Q15.16

$$\begin{aligned}
 f_C &= f_A \times f_B \\
 &= (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \\
 &= (I_A \times I_B) \times 2^{-32}
 \end{aligned}
 \quad \left. \vphantom{\begin{aligned} f_C &= f_A \times f_B \\ &= (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \\ &= (I_A \times I_B) \times 2^{-32} \end{aligned}} \right\} I_C = (I_A \times I_B) \times 2^{-16}$$

$$f_C = I_C \times 2^{-16}$$



Fixed-point Multiplication in UQ16.16



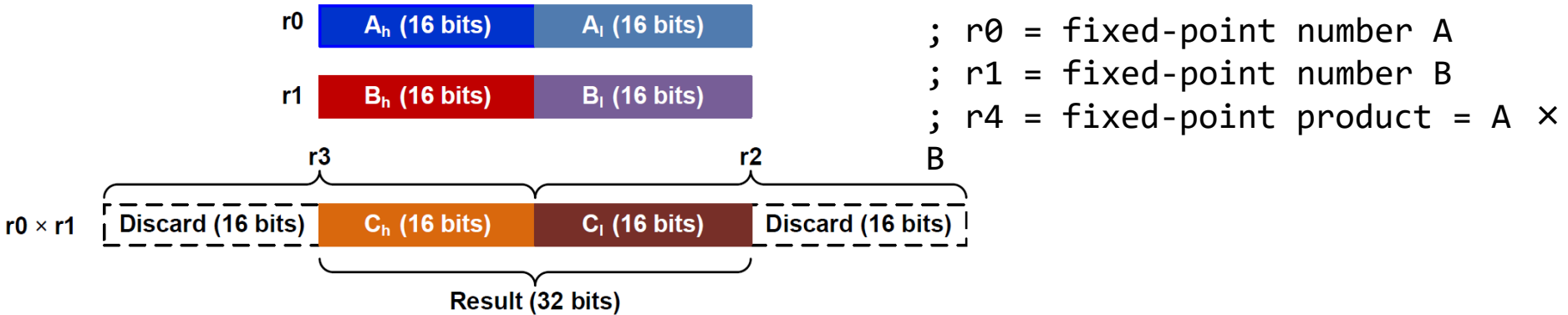
```
UMULL r2, r3, r0, r1    ; Unsigned long multiply
                        ; r2 = low word, r3 = high word
```

```
LSLS r3, r3, #16        ; Shift left high word
```

```
LSRS r2, r2, #16        ; Shift right low word
```

```
ORR r4, r2, r3          ; Pack two halfwords into a word
```

Fixed-point Multiplication in Q15.16



```
SMULL r2, r3, r0, r1 ; Signed Multiply Long
; r2 = low word, r3 = high word
```

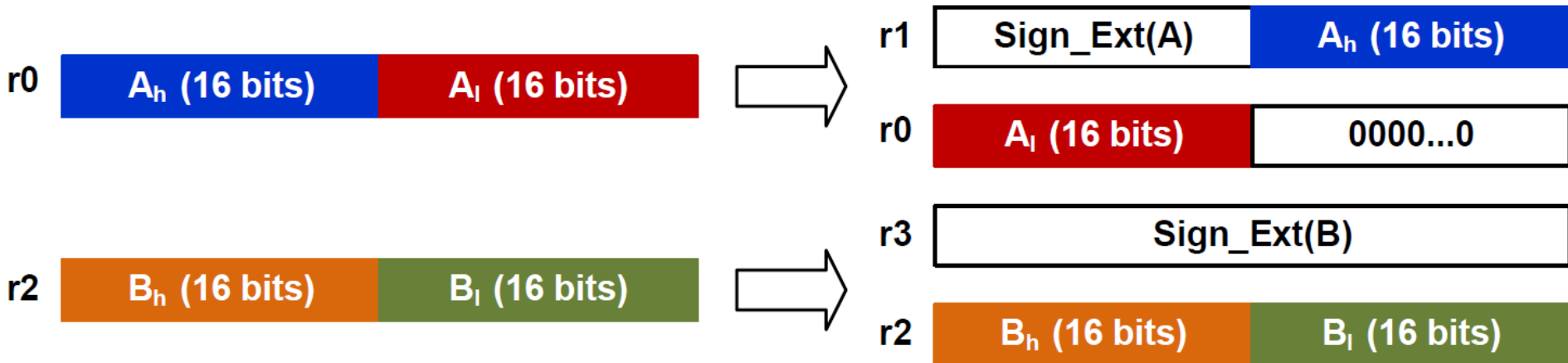
```
LSLS r3, r3, #16 ; Shift left high word
```

```
LSRS r2, r2, #16 ; Shift right low word
```

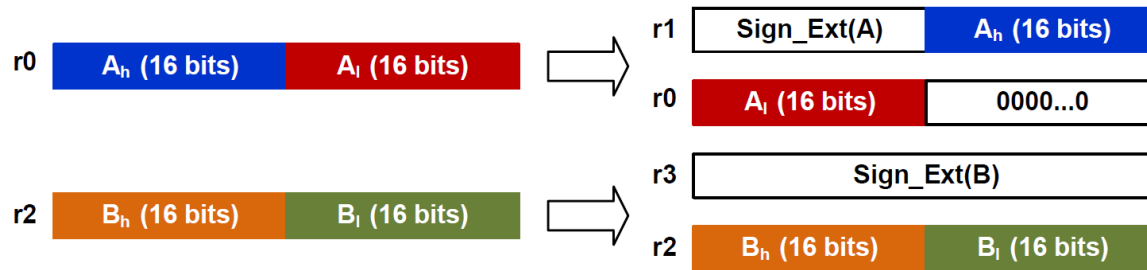
```
ORR r4, r2, r3 ; Pack two halfwords into a word
```

Fixed-point Division in UQ16.16 or Q15.16

$$\begin{aligned}
 f_C &= f_A \div f_B \\
 &= (I_A \times 2^{-16}) \div (I_B \times 2^{-16}) \\
 &= I_A \div I_B \\
 f_C &= I_C \times 2^{-16}
 \end{aligned}
 \quad \left. \begin{array}{l} \\ \\ \\ \end{array} \right\} I_C = (I_A \div I_B) \times 2^{16} = (I_A \times 2^{16}) \div I_B$$



Fixed-point Division



$r4 = \text{fixed-point quotient}$
 $= A \div B$

ASRS r1, r0, #31

; r1[31:0] = sign bits of A

LSLS r1, r1, #16

; put sign bits in upper half word

ORR r1, r1, r0, LSR #16

LSLS r0, r0, #16

; [r1:r0] stores sign extended A

ASRS r3, r2, #31

; [r3:r2] stores sign extended B

BL *division_64_bits*

; Take four input register (r1:r0, r3:r2)

MOV r4, r0

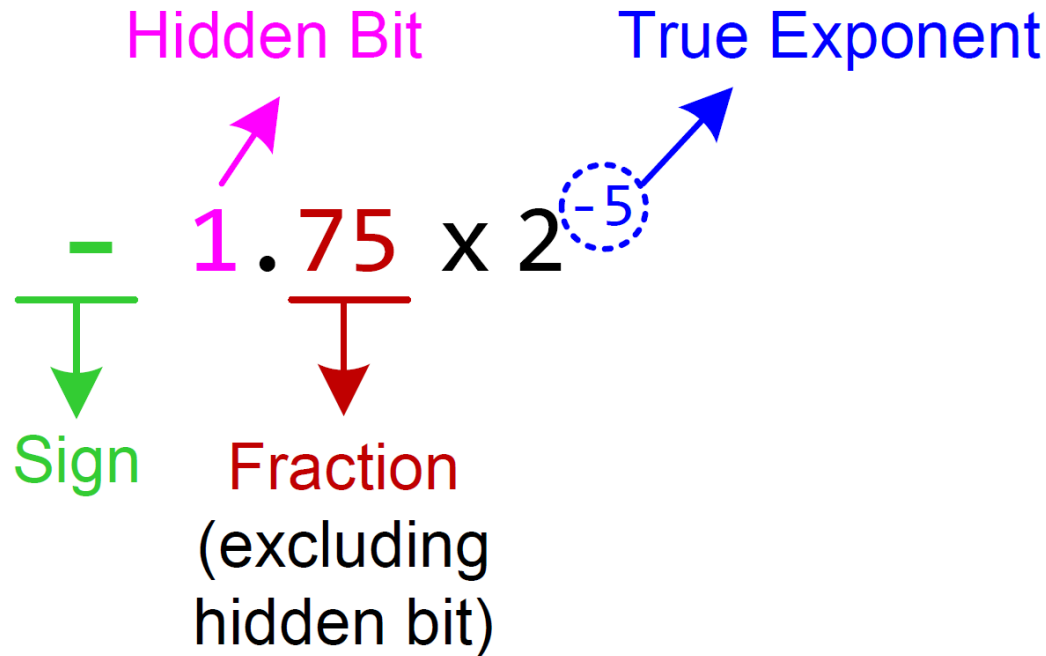
; division_64_bits places result in r0

Floating-Point Numbers

There's no sense in being precise when you don't even know what you're talking about.

John von Neumann
mathematician, computer scientist

Normalization

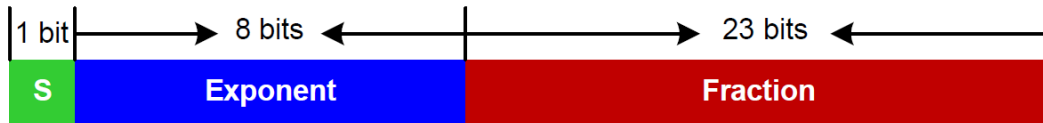


We can convert 10.746×2^6 to normalized format as follows:

$$10.746 \times 2^6 = \frac{10.746}{8} \times 8 \times 2^6 = 1.34325 \times 2^9$$

IEEE Std 754

Single Precision (32 bits)



Double Precision (64 bits)



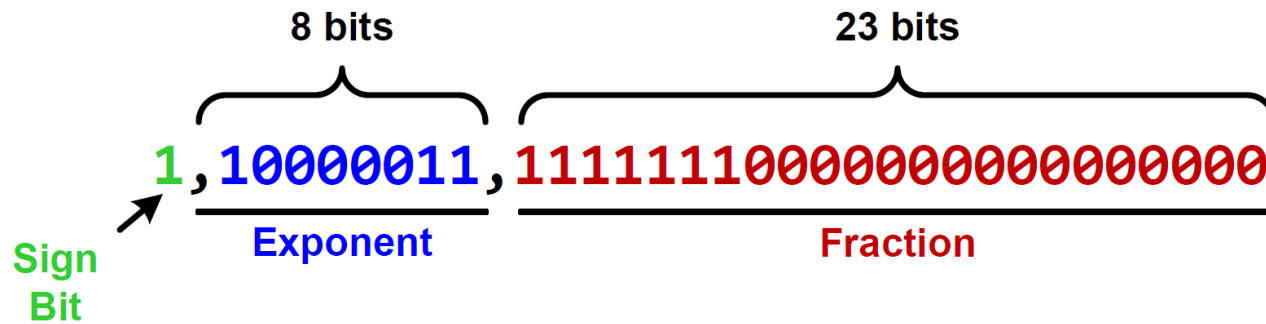
IEEE 754 value:

$$(-1)^S \times (1 + \textit{Fraction}) \times 2^{\textit{Exponent} - \textit{Bias}}$$

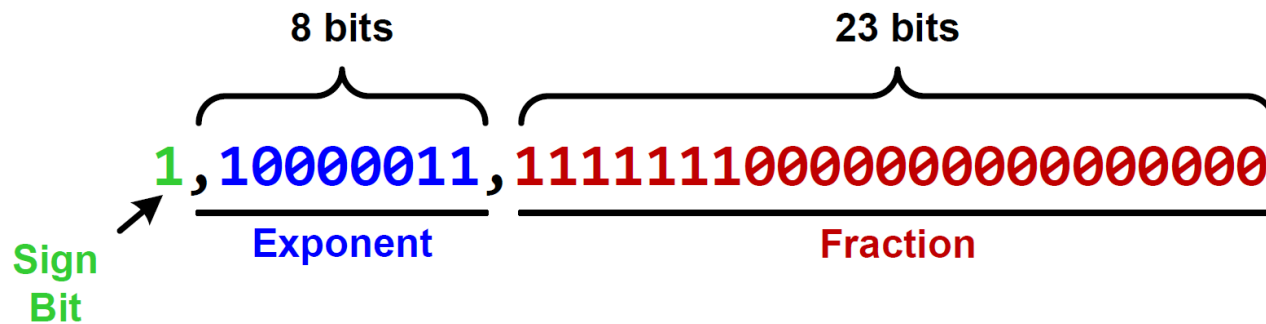
where Bias = **127** for single precision

Bias = **1023** for double precision

Decoding **0xC1FF0000** into a floating-point number

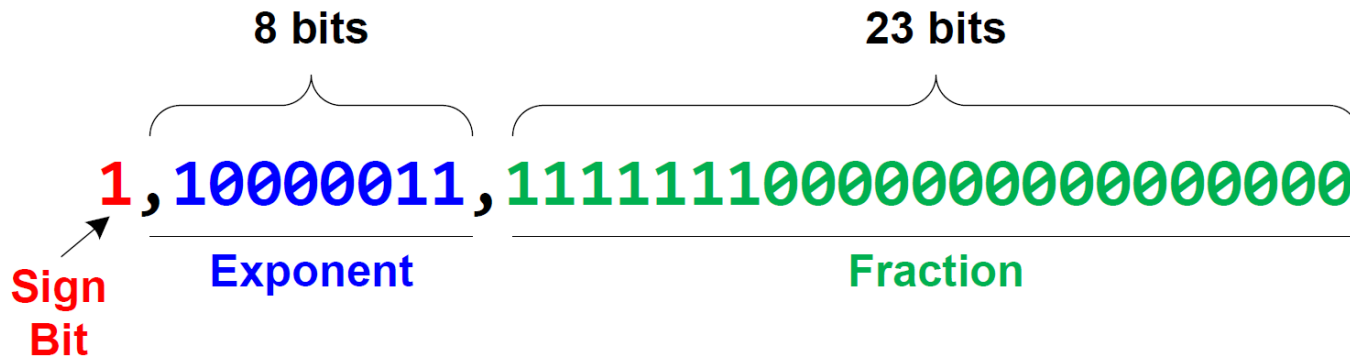


Decoding **0xC1FF0000** into a floating-point number



- Sign = 1
- Exponent = $10000011_2 = 131$
- Fraction = 0.1111111_2

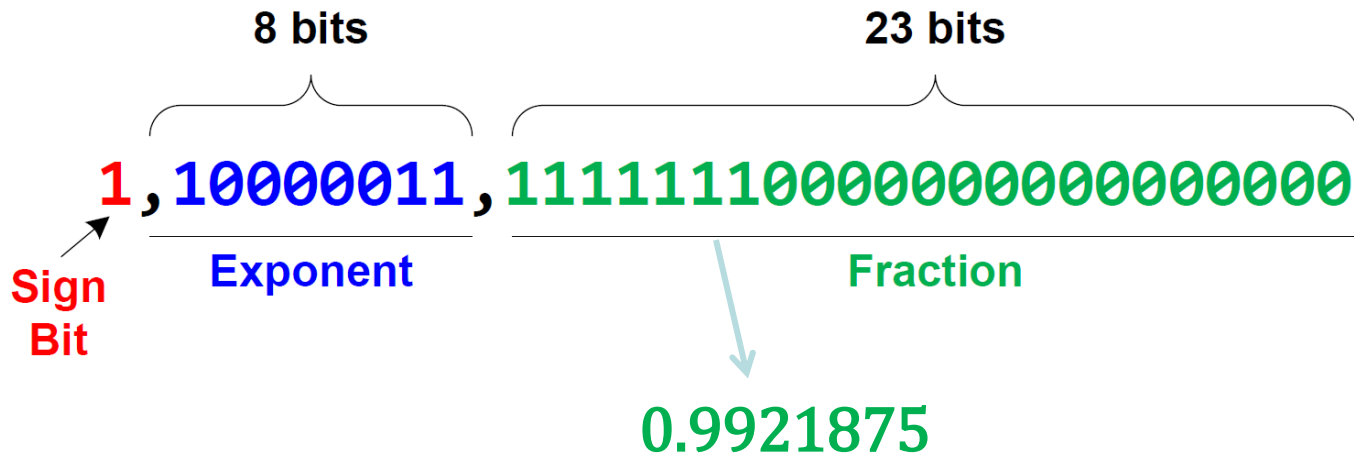
Decoding **0xC1FF0000** into a floating-point number



Fraction

$$\begin{aligned} 0.1111111_2 &= 1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-6} + 1 \times 2^{-7} \\ &= 0.5 + 0.25 + 0.125 + 0.0625 + 0.03125 + 0.015625 + 0.0078125 \\ &= 0.9921875 \end{aligned}$$

Decoding **0xC1FF0000** into a floating-point number



$$\begin{aligned} f &= (-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent} - 127} \\ &= (-1)^1 \times (1 + 0.9921875) \times 2^{131 - 127} \\ &= -1 \times 1.9921875 \times 2^4 \\ &= -31.875 \end{aligned}$$

Encoding **14.5** into IEEE Std 754 Single-Precision

Normalization:

$$\begin{aligned} 14.5 &= 1.8125 \times 2^3 \\ &= (1 + 0.8125) \times 2^3 \end{aligned}$$

- *Sign* = 0
- *Exponent* = $3 + 127 = 130 = 1000010_2$
- *Fraction* = 0.1101_2
 - $0.8125 \times 2 = 1.625 = 1 + 0.625$
 - $0.625 \times 2 = 1.25 = 1 + 0.25$
 - $0.25 \times 2 = 0.5 = 0 + 0.5$
 - $0.5 \times 2 = 1$
- $14.5 = 01000001011010000000000000000000$ in binary or 0x41680000 in hex

Encoding **0.375** into IEEE Std 754 Single-Precision

Consider a value of 0.375. We saw that $0.375 = (0.011)_2 = (1.1)_2 \times 2^{-2}$

Hence after determining a representation of 0.375 as $(1.1)_2 \times 2^{-2}$ we can proceed as above:

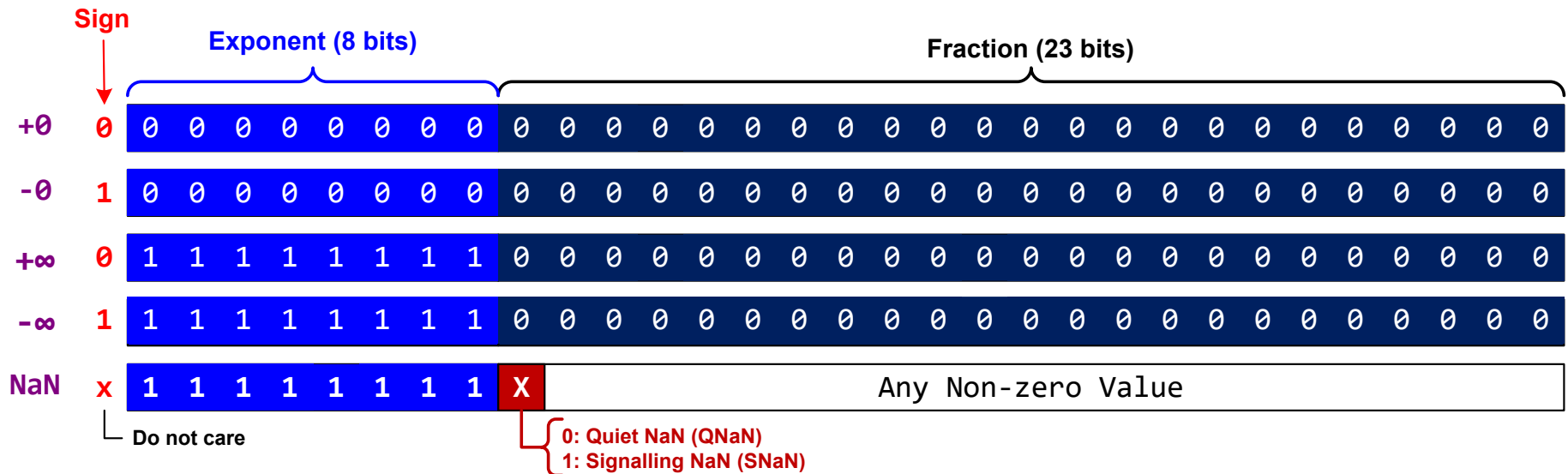
- The exponent is -2 (and in the biased form it is $(127 + (-2))_{10} = (125)_{10} = (0111\ 1101)_2$)
- The fraction is 1 (looking to the right of binary point in 1.1 is a single $1 = x_1$)

From these we can form the resulting 32-bit IEEE 754 binary32 format representation of real number 0.375:

$$(0.375)_{10} = (0\ 01111101\ 100000000000000000000000)_2 = (3EC00000)_{16}$$

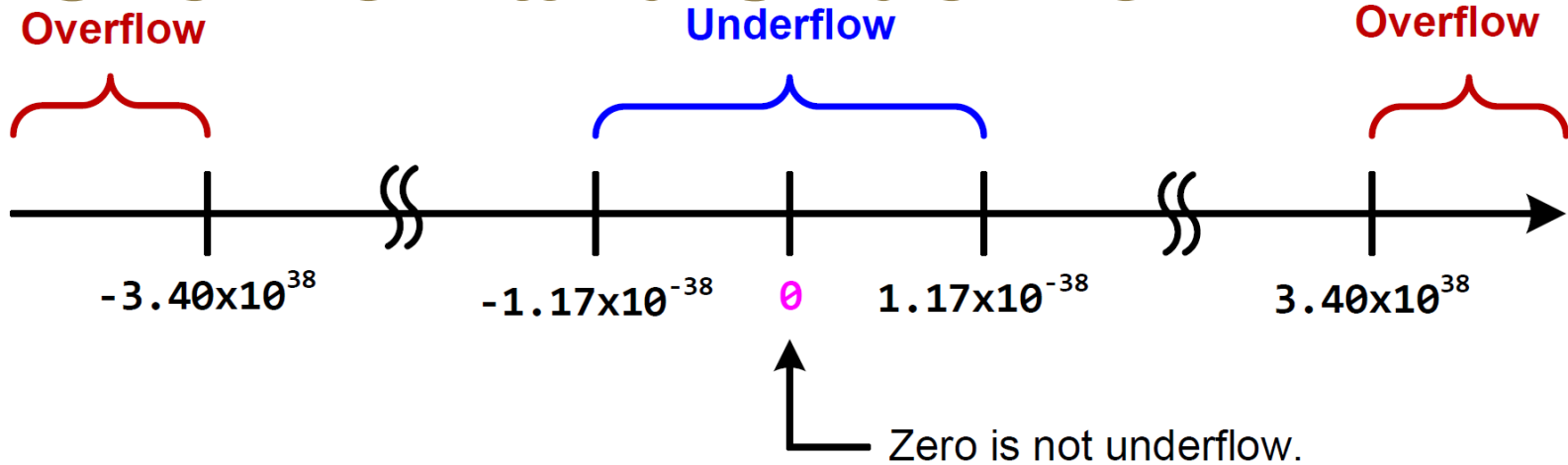
Special Values

- Exponents 00000000 and 11111111 are reserved.



- Example of **Not-A-Number** (NaN)
 - $\log(-10.0)$, $\sqrt{-1.0}$, $0.0/0.0$, $-\infty + \infty$,

Overflow and Underflow

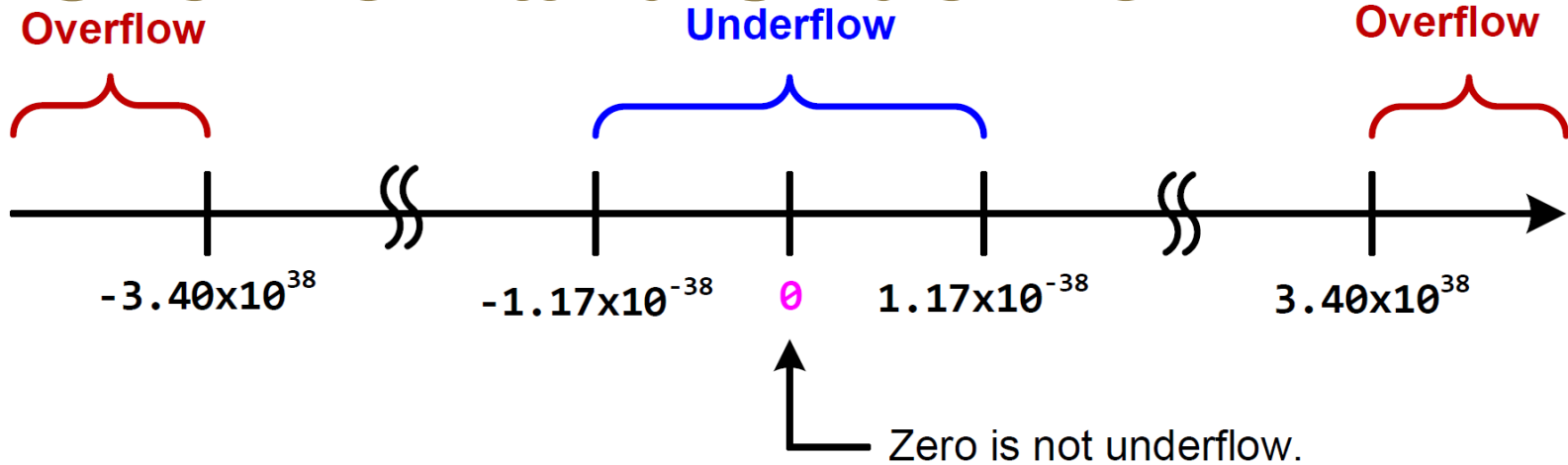


$$(-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Numbers closest to zero

$$(-1)^S \times (1 + 0) \times 2^{1-127} = \pm 2^{-126} \approx \pm 1.18 \times 10^{-38}$$

Overflow and Underflow



$$(-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-127}$$

Numbers farthest from zero

$$(-1)^S \times \left(1 + (1 - 2^{-23})\right) \times 2^{254-127} = \pm(2^{128} - 2^{104}) \approx \pm 3.40 \times 10^{38}$$

Subnormalized Float Number

- To represent numbers between 0 and the minimum positive number that the normalized format can represent.
- Normalized Format

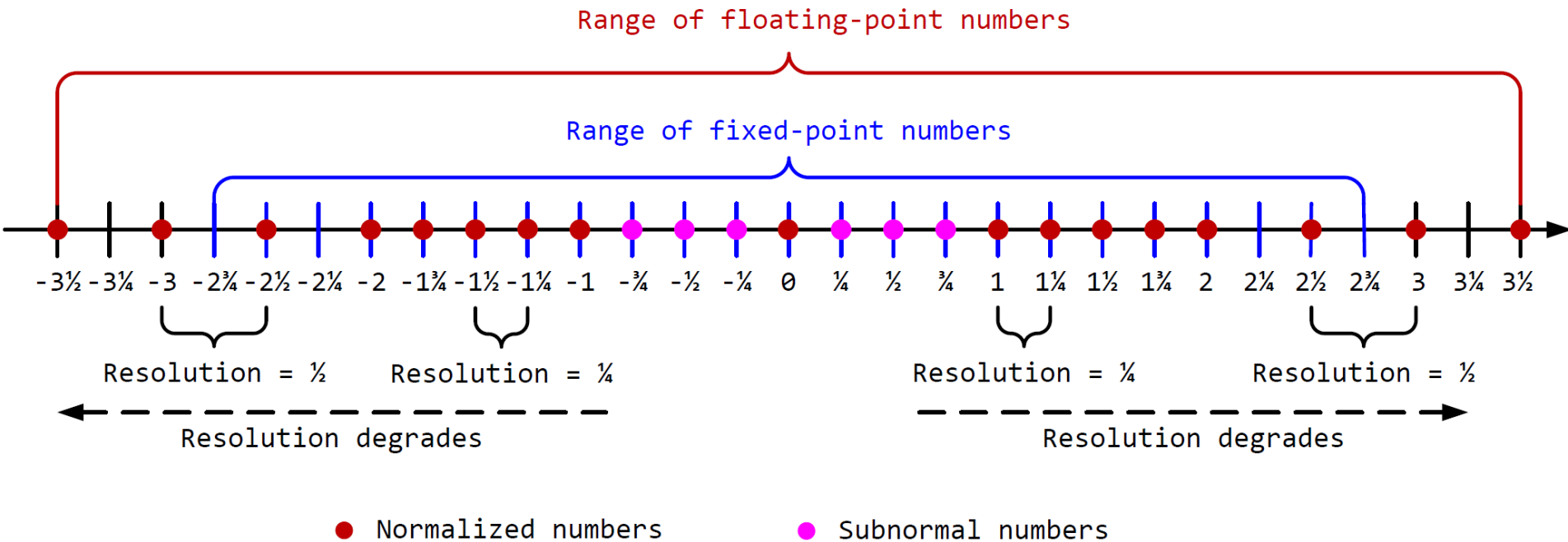
$$(-1)^S \times (1 + Fraction) \times 2^{\text{Exponent}-127}$$

- **Sub-normalized** Format

$$(-1)^S \times Fraction \times 2^{-126}$$

Resolution

- Consider a hypothetical five-bit floating-point system (similar to the IEEE 754 standard).
 - the sign bit, an exponent (2 bits), and a fraction (2 bits)



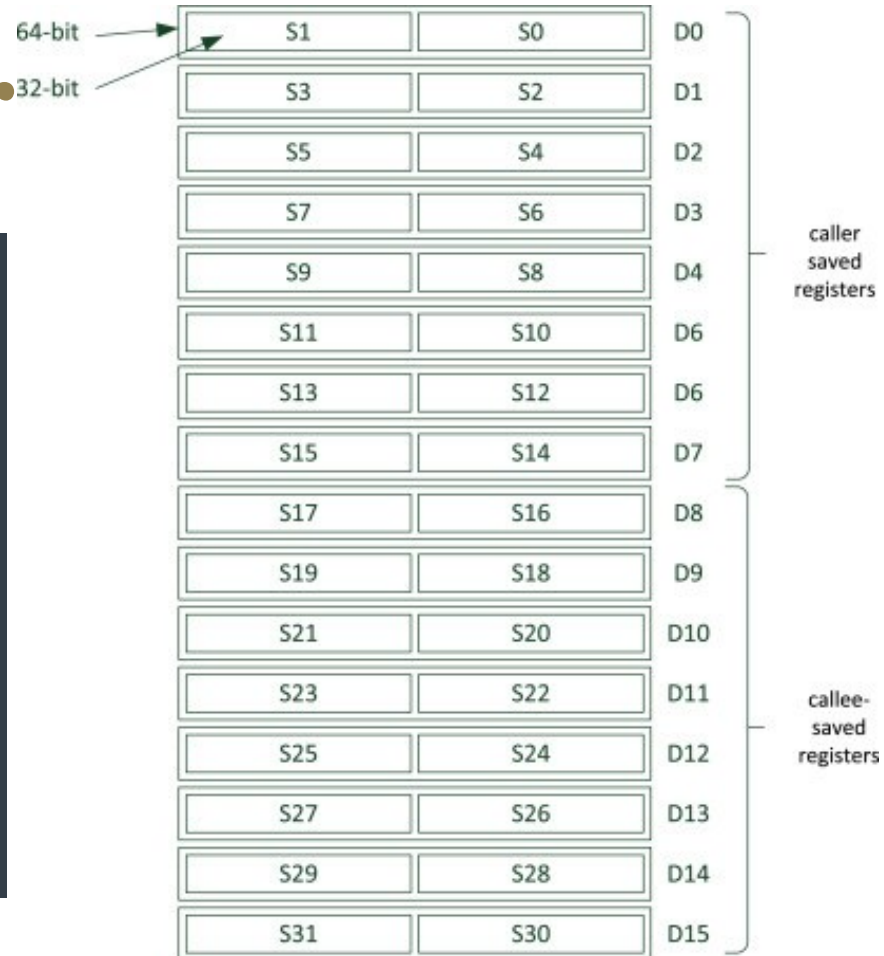
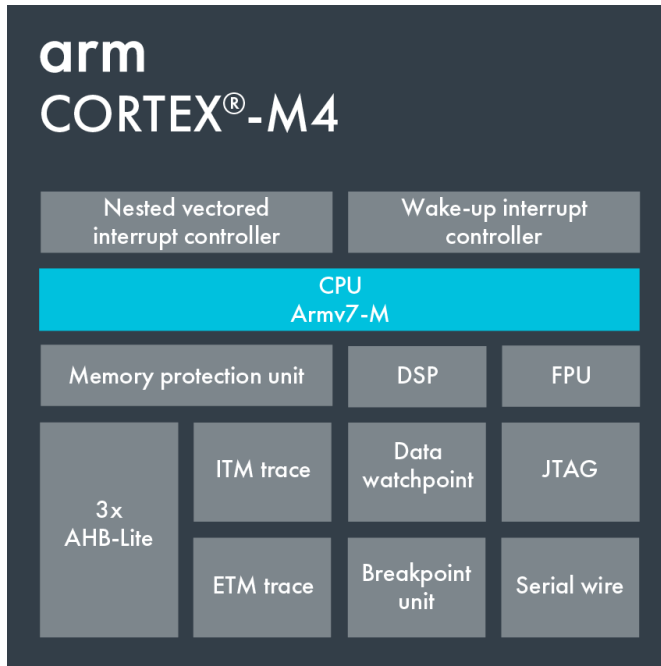
Rounding

- IEEE 754 defines four different rounding rules:
 - round to the nearest value,
 - round toward zero (truncation),
 - round toward plus infinity (rounding up), and
 - round toward negative infinity (rounding down).

Assume only 5 digits are allowed after the decimal point

Rounding Rule	Data	Rounded Result
Nearest	+0.123456	+0.12346
	-0.123456	-0.12346
Truncate	+0.123456	+0.12345
	-0.123456	-0.12345
Rounding up	+0.123456	+0.12346
	-0.123456	-0.12345
Rounding down	+0.123456	+0.12345
	-0.123456	-0.12346

FPU Register



FPSCR format								Rounding		(Stride - 1)*3		Vector length - 1				Exception trap enable bits					Cumulative exception bits										
31	30	29	28	27	26	25	24	23	22	21	20	18	17	16	15			12	11	10	9	8	7			4	3	2	1	0	
N	Z	C	V	QC	AHP	DB	FZ	RMODE]		STRIDE		LEN			IDE			IXE	UFE	OFE	DZE	IOE	IDC			IXC	UFC	OFC	DZC	IOC	
FZ: 1 = flush to zero mode.								Rounding: 0 = round to nearest, 1 = towards +∞, 2 = towards -∞, 3 = towards zero.								(Vector length * Stride) must not exceed 4 for double precision operands. (Deprecated)															

FZ: 1 = flush to zero mode.

Rounding: 0 = round to nearest, 1 = towards $+\infty$, 2 = towards $-\infty$, 3 = towards zero.

(Vector length * Stride) must not exceed 4 for double precision operands. (Deprecated)

FPU Instructions

Operation	§	Assembler	Exceptions	Action
Vector arithmetic	Multiply	VMUL{C} .<P> Fd, Fn, Fm	IO, OF, UF, IX	Fd := Fn * Fm
	and negate	VNMUL{C} .<P> Fd, Fn, Fm	IO, OF, UF, IX	Fd := - (Fn * Fm)
	and accumulate	VMLA{C} .<P> Fd, Fn, Fm	IO, OF, UF, IX	Fd := Fd + (Fn * Fm)
	negate and accumulate	VMLS{C} .<P> Fd, Fn, Fm	IO, OF, UF, IX	Fd := Fd - (Fn * Fm)
	and subtract	VNMLS{C} .<P> Fd, Fn, Fm	IO, OF, UF, IX	Fd := - Fd + (Fn * Fm)
	negate and subtract	VNMLA{C} .<P> Fd, Fn, Fm	IO, OF, UF, IX	Fd := - Fd - (Fn * Fm)
	Add	VADD{C} .<P> Fd, Fn, Fm	IO, OF, IX	Fd := Fn + Fm
	Subtract	VSUB{C} .<P> Fd, Fn, Fm	IO, OF, IX	Fd := Fn - Fm
	Divide	VDIV{C} .<P> Fd, Fn, Fm	IO, DZ, OF, UF, IX	Fd := Fn / Fm
	Absolute	VABS{C} .<P> Fd, Fm		Fd := abs(Fm)
	Negative	VNEG{C} .<P> Fd, Fm		Fd := - Fm
	Square root	VSQRT{C} .<P> Fd, Fm	IO, IX	Fd := sqrt(Fm)

Key to Tables

{C}

See Table **Condition Field**

<P>

F32 (single precision) or F64 (double precision).

S, D, H

Single, double, or half-precision (F16).

FPU Instructions

Scalar compare	Two values Value with zero	VCMP{E}{C}.<P> Fd, Fm VCMP{E}{C}.<P> Fd, #0.0	IO IO	Set FPSCR flags on Fd – Fm Set FPSCR flags on Fd – 0	Use VMRS APSR_nzcv, FPSCR to transfer flags.
Scalar convert	Single to double Double to single Unsigned integer to float Signed integer to float Float to unsigned integer Float to signed integer Fixed-point to float Float to fixed-point Single to half-precision Single to half-precision Half to single-precision Half to single-precision	VCVT{C}.F64.F32 Dd, Sm VCVT{C}.F32.F64 Sd, Dm VCVT{C}.<P>.U32 Fd, Sm VCVT{C}.<P>.S32 Fd, Sm VCVT{R}{C}.U32.<P> Sd, Fm VCVT{R}{C}.S32.<P> Sd, Fm 3 VCVT{C}.<P>.<type> Fd, Fd, #<fbits> 3 VCVT{C}.<type>.<P> Fd, Fd, #<fbits> 3H VCVTT{C}.F16.F32 Sd, Sm 3H VCVTB{C}.F16.F32 Sd, Sm 3H VCVTT{C}.F32.F16 Sd, Sm 3H VCVTB{C}.F32.F16 Sd, Sm	IO IO, OF, UF, IX IX IX IO, IX IO, IX IO, IX IO, IX ID, IO, OF, UF, IX ID, IO, OF, UF, IX ID, IO, OF, UF, IX ID, IO, OF, UF, IX	Dd := convertStoD(Sm) Sd := convertDtoS(Dm) Fd := convertUItoF(Sm) Fd := convertSItoF(Sm) Sd := convertFtoUI(Fm) Sd := convertFtoSI(Fm) Fd := convert<type>toF(Fd) Fd := convertFto<type>(Fd) Sd:=convertStoH(Sm) Sd:=convertStoH(Sm) Sd:=convertHtoS(Sm) Sd:=convertHtoS(Sm)	Source is in bottom 16 or 32 bits of Fd. Destination is bottom 16 or 32 bits of Fd. Destination is top 16 bits of Sd Destination is bottom 16 bits of Sd Source is top 16 bits of Sm Source is bottom 16 bits of Sm
Insert constant	Insert constant in register	3 VMOV{C}.<P> Fd, #<fpconst>		Fd := <fpconst>	
Transfer registers	Copy VFP register ARM® to single Single to ARM Two ARM to two singles Two singles to two ARM Two ARM to double Double to two ARM ARM to lower half of double Lower half of double to ARM	VMOV{C}.<P> Fd, Fm VMOV{C} Sn, Rd VMOV{C} Rd, Sn 2 VMOV{C} Sn, Sm, Rd, Rn 2 VMOV{C} Rd, Rn, Sn, Sm 2 VMOV{C} Dm, Rd, Rn 2 VMOV{C} Rd, Rn, Dm VMOV{C} Dn[0], Rd VMOV{C} Rd, Dn[0]		Fd := Fm Sn := Rd Rd := Sn Sn := Rd, Sm := Rn Rd := Sn, Rn := Sm Dm[31:0] := Rd, Dm[63:32] := Rn Rd := Dm[31:0], Rn := Dm[63:32] Dn[31:0] := Rd Rd := Dn[31:0]	Sm must be S(n+1) Sm must be S(n+1)
Operation		\$ Assembler	Exceptions	Action	Notes
Transfer registers (continued)	ARM to upper half of double Upper half of double to ARM ARM to VFP system register VFP system register to ARM FPSCR flags to APSR	VMOV{C} Dn[1], Rd VMOV{C} Rd, Dn[1] VMRS{C} <VFPsysreg>, Rd VMRS{C} Rd, <VFPsysreg> VMRS{C} APSR_nzcv, FPSCR		Dn[63:32] := Rd Rd := Dn[63:32] VFPsysreg := Rd Rd := VFPsysreg APSR flags := FPSCR flags	

FPU Instructions

Operation		§	Assembler	Synonyms	Action
Save VFP registers	Single Single, PC-relative Multiple, unindexed / increment after decrement before Push onto stack		VSTR{C} Fd, [Rn{, #<immed>}] VSTR{C} Fd, <label> VSTM{C} Rn{!}, <VFPregs> VSTMDB{C} Rn!, <VFPregs> VPUSH{C} <VFPregs>	VSTMIA, VSTMEA VSTMFD (full descending) VSTMFD SP!	[address] := Fd. Immediate range 0-1020, multiple of 4. Saves list of VFP registers, starting at address in Rn.
Load VFP registers	Single Single, PC-relative Multiple, unindexed / increment after decrement before Pop from stack		VLDR{C} Fd, [Rn{, #<immed>}] VLDR{C} Fd, <label> VLDM{C} Rn{!}, <VFPregs> VLDMDB{C} Rn!, <VFPregs> VPOP{C} <VFPregs>	VLDMIA, VLDMFD VLDMEA (empty ascending) VLDM SP!	Fd := [address]. Immediate range 0-1020, multiple of 4. Loads list of VFP registers, starting at address in Rn.